# Software Requirements Specification

for

# NoteFlow

Collaborative Educational Resource Sharing Platform

Version 1.0 approved

Prepared by

Team 27

24CSB0B84, 24CSB0B85, 24CSB0B86

Computer Science Department

National Institute of Technology, Warangal

16 February 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

# Contents

**1 Introduction** **3**
1.1 Purpose . . . 3
1.2 Document Conventions . . . 3
1.3 Intended Audience and Reading Suggestions . . . 3
1.4 Product Scope . . . 4
1.5 References . . . 4

**2 Overall Description** **5**
2.1 Product Perspective . . . 5
2.2 Product Functions . . . 5
2.3 User Classes and Characteristics . . . 6
2.4 Operating Environment . . . 7
2.5 Design and Implementation Constraints . . . 7
2.6 User Documentation . . . 8
2.7 Assumptions and Dependencies . . . 8

**3 External Interface Requirements** **10**
3.1 User Interfaces . . . 10
3.1.1 Dashboard Interface . . . 10
3.1.2 Classroom Interface . . . 10
3.1.3 PDF Viewer Interface . . . 10
3.1.4 Analytics Dashboard Interface . . . 11
3.2 Hardware Interfaces . . . 11
3.3 Software Interfaces . . . 12
3.4 Communications Interfaces . . . 13

**4 System Features** **15**
4.1 User Authentication and Authorization . . . 15
4.1.1 Description and Priority . . . 15
4.1.2 Stimulus/Response Sequences . . . 15
4.1.3 Functional Requirements . . . 15
4.2 Classroom Management . . . 16
4.2.1 Description and Priority . . . 16
4.2.2 Stimulus/Response Sequences . . . 16
4.2.3 Functional Requirements . . . 16
4.3 Syllabus Tree Navigation . . . 17
4.3.1 Description and Priority . . . 17
4.3.2 Stimulus/Response Sequences . . . 17
4.3.3 Functional Requirements . . . 17
4.4 File Upload and Management . . . 18
4.4.1 Description and Priority . . . 18
4.4.2 Stimulus/Response Sequences . . . 18
4.4.3 Functional Requirements . . . 19
4.5 File Version Control . . . 19
4.5.1 Description and Priority . . . 19
4.5.2 Stimulus/Response Sequences . . . 19
4.5.3 Functional Requirements . . . 20

4.6 In-Browser PDF Viewer with Lazy Loading . . . 20
4.6.1 Description and Priority . . . 20
4.6.2 Stimulus/Response Sequences . . . 20
4.6.3 Functional Requirements . . . 21
4.7 Collaborative Heatmap Visualization . . . 21
4.7.1 Description and Priority . . . 21
4.7.2 Stimulus/Response Sequences . . . 21
4.7.3 Functional Requirements . . . 22
4.8 Discussion Threads . . . 22
4.8.1 Description and Priority . . . 22
4.8.2 Stimulus/Response Sequences . . . 22
4.8.3 Functional Requirements . . . 23
4.9 Exam Resources Section . . . 23
4.9.1 Description and Priority . . . 23
4.9.2 Stimulus/Response Sequences . . . 23
4.9.3 Functional Requirements . . . 24
4.10 Revision Tests . . . 24
4.10.1 Description and Priority . . . 24
4.10.2 Stimulus/Response Sequences . . . 25
4.10.3 Functional Requirements . . . 25
4.11 Analytics Dashboard (Professor) . . . 26
4.11.1 Description and Priority . . . 26
4.11.2 Stimulus/Response Sequences . . . 26
4.11.3 Functional Requirements . . . 26

**5 Other Nonfunctional Requirements 28**
5.1 Performance Requirements . . . 28
5.2 Safety Requirements . . . 28
5.3 Security Requirements . . . 28
5.4 Software Quality Attributes . . . 29
5.4.1 Usability . . . 29
5.4.2 Reliability . . . 29
5.4.3 Scalability . . . 29
5.4.4 Maintainability . . . 29
5.4.5 Portability . . . 30
5.4.6 Interoperability . . . 30
5.5 Business Rules . . . 30

**6 Other Requirements 31**
6.1 Database Requirements . . . 31
6.2 Internationalization Requirements . . . 31
6.3 Legal Requirements . . . 31
6.4 Installation and Deployment Requirements . . . 31

# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| Team27 | February 16, 2026 | Initial document creation | 1.0 |

# 1 Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the NoteFlow platform, a collaborative educational resource sharing system. This document is intended for developers, project managers, testers, and stakeholders involved in the development and deployment of NoteFlow Release 1.0.

NoteFlow is designed to transform educational resource management from traditional file-based chaos into an organized, collaborative ecosystem where students and professors can efficiently share, organize, and enhance learning materials.

## 1.2 Document Conventions

This document follows the IEEE Standard 830-1998 for Software Requirements Specifications. The following conventions are used throughout:

- **Requirement IDs**: Each requirement is uniquely identified with a prefix indicating its category (e.g., FR-1 for Functional Requirement 1, NFR-1 for Non-Functional Requirement 1)
- **Priority Levels**: Requirements are categorized as High, Medium, or Low priority
- **Shall/Should/May**: "Shall" indicates mandatory requirements, "should" indicates recommended features, "may" indicates optional features
- Higher-level requirements inherit priorities to detailed sub-requirements unless explicitly stated otherwise

## 1.3 Intended Audience and Reading Suggestions

**Developers**: Focus on Sections 3 (External Interface Requirements), 4 (System Features), and 5 (Other Nonfunctional Requirements) for implementation details.

**Project Managers**: Review Section 1 for project scope and Section 2 for system overview. Section 4 provides feature priorities for planning.

**Testers**: Concentrate on Section 4 (System Features) for test case development and Section 5 for performance and security testing requirements.

**Users (Students & Professors)**: Section 2 (Overall Description) provides the best overview of system capabilities and user experience.

**Database Administrators**: Review Section 3.3 (Software Interfaces) for database requirements and data management specifications.

## 1.4 Product Scope

NoteFlow addresses critical challenges in educational resource management by providing a centralized, collaborative platform that eliminates organizational chaos, enables structured resource sharing, and provides actionable insights through analytics.

**Benefits and Objectives:**

- Reduce resource discovery time by 40% through syllabus-aligned organization
- Increase student engagement through collaborative features and gamification
- Provide professors with data-driven insights into learning patterns
- Ensure resource quality through verification and version control
- Foster a collaborative learning community with reputation systems

**Business Strategy Alignment:** NoteFlow aligns with the growing trend toward digital education transformation and collaborative learning methodologies, addressing the need for intelligent resource management in higher education institutions.

## 1.5 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
2. React.js Documentation: `https://react.dev`
3. Node.js Documentation: `https://nodejs.org/docs`
4. PostgreSQL Documentation: `https://www.postgresql.org/docs/`
5. Supabase Documentation: `https://supabase.com/docs`
6. NoteFlow Use Cases Document, Version 1.0, January 2026
7. NoteFlow Project Report, January 2026

# 2 Overall Description

## 2.1 Product Perspective

NoteFlow is a new, self-contained web-based platform designed specifically for educational resource management. Unlike traditional Learning Management Systems (LMS) that serve as basic file repositories, NoteFlow creates an intelligent, collaborative ecosystem.

**System Context:** NoteFlow operates as a standalone web application accessible through modern browsers on desktop and mobile devices. It integrates with cloud storage services (Supabase Storage) for file management and uses PostgreSQL (Neon DB) for data persistence.

**Major System Components:**

**Three-Tier Architecture:**

1. **Presentation Layer (Client Side):**
   - Web Browser Interface
   - React.js Frontend Application
   - User Interface Components
   - PDF Viewer with Heatmap Overlay
   - Syllabus Tree Navigation
2. **Application Layer (Server Side):**
   - Node.js/Express Backend
   - Authentication Service
   - File Processing Service
   - Heatmap Aggregation Service
   - Communication via HTTPS/REST API
3. **Data Layer:**
   - PostgreSQL Database (Neon DB) for structured data
   - Supabase Storage for file storage and CDN delivery

## 2.2 Product Functions

NoteFlow provides the following major functions:

- **Classroom Management**: Professors create private classrooms with unique 6-digit invite codes; students join classrooms using these codes
- **Syllabus Tree Organization**: Interactive hierarchical syllabus structure with drag-and-drop editing for organizing course content
- **Smart File Management**: Upload files with automatic thumbnail generation, virus scanning, compression, and metadata tagging mapped to syllabus topics

- **Version Control**: Track and manage multiple versions of educational resources with rollback capabilities
- **In-Browser PDF Viewer**: View PDFs with lazy loading, zoom, navigation, and annotation capabilities
- **Collaborative Heatmap**: Visualize aggregated student highlights to identify exam-critical content
- **Discussion Threads**: Contextual Q&A discussions linked to specific resources and topics
- **Bounty Board**: Request and incentivize creation of missing resources
- **Analytics Dashboard**: Comprehensive insights for professors on engagement patterns, resource usage, and learning gaps

## 2.3 User Classes and Characteristics

**User Class 1: Professor**

- **Frequency of Use**: Daily during semester, 2-3 hours per week
- **Technical Expertise**: Moderate (comfortable with web applications)
- **Key Functions**: Create classrooms, manage syllabus structure, verify student uploads, access analytics, moderate discussions
- **Characteristics**: Requires administrative control, needs visibility into student engagement, values quality control
- **Importance**: Critical user class - system functionality depends on professor setup

**User Class 2: Student**

- **Frequency of Use**: Daily during active study periods, especially before exams
- **Technical Expertise**: Basic to moderate (familiar with web browsing and file uploads)
- **Key Functions**: Join classrooms, upload resources, view documents with heatmaps, participate in discussions, take revision tests, earn karma points
- **Characteristics**: Seeks efficient resource discovery, values peer collaboration, motivated by gamification
- **Importance**: Primary user class - system success measured by student engagement

## 2.4 Operating Environment

NoteFlow operates in the following environment:

**Client-Side:**

- Modern web browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)
- Desktop operating systems: Windows 10/11, macOS 10.15+, Linux
- Mobile operating systems: iOS 14+, Android 10+
- Minimum screen resolution: 1024x768 (desktop), 375x667 (mobile)
- JavaScript enabled
- Stable internet connection (minimum 2 Mbps recommended)

**Server-Side:**

- Node.js runtime version 18.x or higher
- PostgreSQL 14.x database (Neon DB managed service)
- Supabase services for authentication and storage
- Cloud hosting platform (Vercel, Netlify, or equivalent)

## 2.5 Design and Implementation Constraints

**Regulatory and Policy Constraints:**

- Must comply with university email verification requirements for user registration
- Educational content must not violate copyright laws
- User data must be handled according to privacy regulations
- Uploaded files must be scanned for malware before storage

**Hardware Limitations:**

- Maximum file upload size: 100MB per file
- Database storage optimized for PostgreSQL query performance
- Client-side PDF rendering limited by browser memory capabilities

**Technology Constraints:**

- Frontend must use React.js framework for component-based architecture
- Backend must use Node.js with Express.js for REST API
- Database must be PostgreSQL (Neon DB) for ACID compliance
- Authentication must use Supabase Auth for security

- File storage must use Supabase Storage with CDN delivery
- PDF rendering must use PDF.js library for browser compatibility

**Interface Requirements:**

- All API endpoints must follow RESTful design principles
- Communication must use HTTPS protocol with TLS 1.2 or higher
- API responses must use JSON format
- Frontend must be responsive and mobile-friendly using Tailwind CSS

**Development Standards:**

- Code must follow ESLint standards for JavaScript
- Git version control with feature branch workflow
- Code review required before merging to main branch
- Unit test coverage minimum 70% for critical functions

## 2.6 User Documentation

The following user documentation will be delivered with NoteFlow:

- **User Manual**: Comprehensive guide covering all features for both students and professors (PDF and online HTML format)
- **Quick Start Guide**: Step-by-step tutorial for first-time users covering account creation, joining classrooms, and basic operations
- **In-App Help**: Contextual tooltips and help buttons throughout the interface
- **Video Tutorials**: Short screencasts demonstrating key features (hosted on platform)
- **FAQ Section**: Commonly asked questions and troubleshooting tips
- **API Documentation**: Technical documentation for future integrations (for administrators)

## 2.7 Assumptions and Dependencies

**Assumptions:**

- Users have valid university email addresses for registration
- Users have stable internet connectivity during platform usage
- Students and professors are familiar with basic web applications
- The 6-digit invite code system provides sufficient uniqueness for expected classroom volume

- Professors will actively moderate and verify student-uploaded content
- Educational content uploaded by users does not violate copyright
- Browser local storage is enabled for PDF viewer functionality

**Dependencies:**

- **Supabase Services**: Platform depends on Supabase for authentication and file storage availability
- **Neon DB**: Database service uptime and performance
- **PDF.js Library**: Continued maintenance and browser compatibility
- **Third-Party Libraries**: React.js, Express.js, Tailwind CSS, and other npm packages
- **Cloud Infrastructure**: Hosting platform (Vercel/Netlify) availability
- **Email Service**: University email systems for verification
- **Virus Scanning Service**: External malware detection API

# 3 External Interface Requirements

## 3.1 User Interfaces

### 3.1.1 Dashboard Interface

**UI-1: Main Dashboard**

- Clean, modern layout with navigation sidebar
- Displays list of enrolled classrooms as cards with course name, professor, and last activity
- "Create Classroom" button (visible only to professors)
- "Join Classroom" button with invite code input modal
- User profile section showing karma points, level, and badges
- Responsive design adapting to mobile screens (hamburger menu below 768px width)

### 3.1.2 Classroom Interface

**UI-2: Classroom View**

- Three-panel layout:
  1. Left panel: Interactive syllabus tree with expand/collapse nodes
  2. Center panel: File listing with thumbnails, version indicators, and verification badges
  3. Right panel: Discussion threads and bounty board (collapsible)
- Top navigation bar with classroom name, invite code display, and search functionality
- Upload button with drag-and-drop area overlay
- Filter options: by topic, by contributor, by verification status

### 3.1.3 PDF Viewer Interface

**UI-3: In-Browser PDF Viewer**

- Full-screen modal viewer with dark/light mode toggle
- Navigation controls: previous/next page, page number input, thumbnail sidebar
- Zoom controls: fit-to-width, fit-to-page, custom zoom percentage
- Annotation toolbar: highlight tool with color picker, comment tool
- Heatmap overlay toggle button with intensity legend
- Download and print buttons
- Close button returning to classroom view

### 3.1.4 Analytics Dashboard Interface

**UI-4: Professor Analytics Dashboard**

- Overview cards: total resources, active students, weekly uploads, engagement rate
- Interactive charts:
  1. Resource uploads over time (line chart)
  2. Most active students (bar chart)
  3. Topic coverage heatmap (grid visualization)
  4. Discussion activity (timeline)
- Resource gap analysis table with missing topics highlighted
- Export data button (CSV/PDF options)

**GUI Standards:**

- Consistent color scheme: Primary blue (#3B82F6), Success green (#10B981), Warning amber (#F59E0B), Error red (#EF4444)
- Typography: Inter font family, 14px base font size, 1.5 line height
- Spacing: 4px base unit following 8-point grid system
- Button styles: Primary (filled), Secondary (outlined), Tertiary (text only)
- Icons: Lucide React icon library for consistency
- Loading states: Skeleton screens for content, spinners for actions
- Error messages: Toast notifications positioned top-right, auto-dismiss after 5 seconds
- Keyboard shortcuts: Esc to close modals, Tab for navigation, Enter to submit forms

## 3.2 Hardware Interfaces

NoteFlow is a web-based application with no direct hardware interface requirements. However, the following hardware interactions occur through the browser:

**HW-1: Client Device Storage**

- Browser local storage for caching user preferences and temporary data
- Maximum local storage usage: 10MB per user
- IndexedDB for offline PDF page caching (optional enhancement)

**HW-2: File System Access**

- File upload through browser file picker accessing client device storage
- Supported file types: PDF, DOCX, PPTX, TXT

- File download triggering browser's default download behavior

**HW-3: Display Devices**

- Responsive rendering adapting to screen sizes from 320px to 4K displays
- Touch interface support for tablets and mobile devices
- High-DPI display support for retina screens

## 3.3 Software Interfaces

**SI-1: Neon PostgreSQL Database**

- **Name and Version**: Neon DB (PostgreSQL 14.x compatible)
- **Purpose**: Primary data storage for users, classrooms, resources, highlights, discussions, and analytics
- **Interface Type**: PostgreSQL wire protocol over SSL
- **Data Exchange**: SQL queries via node-postgres client library
- **Key Tables**: users, classrooms, syllabus_nodes, resources, highlights, discussions, bounties, karma_transactions
- **Shared Data**: All application state and user-generated content

**SI-2: Supabase Authentication**

- **Name and Version**: Supabase Auth (latest stable)
- **Purpose**: User authentication, session management, email verification
- **Interface Type**: REST API and JavaScript SDK
- **Data Exchange**: JSON Web Tokens (JWT) for session authentication
- **Authentication Methods**: Email/password, magic link email
- **Security**: OAuth 2.0 protocol, bcrypt password hashing

**SI-3: Supabase Storage**

- **Name and Version**: Supabase Storage (latest stable)
- **Purpose**: File storage for PDFs, documents, thumbnails, and user avatars
- **Interface Type**: REST API and JavaScript SDK
- **Data Exchange**: Multipart form-data for uploads, CDN URLs for retrieval
- **Storage Buckets**: resources (public), thumbnails (public), avatars (public)
- **Access Control**: Row-level security policies based on user roles

**SI-4: PDF.js Library**

- **Name and Version**: PDF.js v3.x (Mozilla)
- **Purpose**: Client-side PDF rendering in browser canvas
- **Interface Type**: JavaScript API
- **Data Exchange**: PDF binary data, canvas rendering commands
- **Functions Used**: Document loading, page rendering, text layer extraction

**SI-5: Virus Scanning Service**

- **Name and Version**: ClamAV or VirusTotal API
- **Purpose**: Malware detection in uploaded files
- **Interface Type**: REST API
- **Data Exchange**: File hash or file upload, scan result JSON
- **Integration**: Server-side scanning before file storage

## 3.4 Communications Interfaces

**CI-1: HTTP/HTTPS Protocol**

- All client-server communication uses HTTPS with TLS 1.2 or higher
- RESTful API endpoints following standard HTTP methods:
  - GET for data retrieval
  - POST for resource creation
  - PUT/PATCH for updates
  - DELETE for removal
- Response codes: 2xx (success), 4xx (client errors), 5xx (server errors)

**CI-2: WebSocket Protocol**

- Real-time updates for discussion threads and collaborative features
- Server-sent events for heatmap data updates
- Fallback to polling for browsers without WebSocket support

**CI-3: Email Communication**

- SMTP protocol for sending verification emails and notifications
- Email templates for: account verification, classroom invitations, approval notifications
- Sender: noreply@noteflow.edu

**CI-4: Data Transfer Formats**

- JSON for API request/response bodies
- Multipart/form-data for file uploads
- URL-encoded for form submissions

**CI-5: Security and Encryption**

- JWT tokens for authentication with 24-hour expiration
- CORS enabled with whitelist of allowed origins
- Rate limiting: 100 requests per minute per IP address
- CSRF protection using double-submit cookie pattern

**CI-6: Data Synchronization**

- Auto-save for PDF annotations every 30 seconds
- Optimistic UI updates with server reconciliation
- Conflict resolution for concurrent edits (last-write-wins)

# 4 System Features

## 4.1 User Authentication and Authorization

### 4.1.1 Description and Priority

**Priority: High**

User authentication is the foundational feature enabling secure access to NoteFlow. The system implements role-based access control (RBAC) distinguishing between Professor and Student roles, each with specific permissions.

### 4.1.2 Stimulus/Response Sequences

1. User navigates to NoteFlow landing page
2. User clicks "Sign Up" button
3. System displays registration form
4. User enters university email, password, and selects role
5. System validates email format and password strength
6. System sends verification email with magic link
7. User clicks verification link in email
8. System activates account and redirects to dashboard

### 4.1.3 Functional Requirements

**FR-1.1**: System shall require university email addresses ending in .edu domain for registration.

**FR-1.2**: System shall enforce password requirements: minimum 8 characters, at least one uppercase, one lowercase, one number, and one special character.

**FR-1.3**: System shall send email verification within 2 minutes of registration.

**FR-1.4**: System shall allow users to select Professor or Student role during registration.

**FR-1.5**: System shall maintain user sessions using JWT tokens valid for 24 hours.

**FR-1.6**: System shall automatically logout users after 24 hours of inactivity.

**FR-1.7**: System shall provide "Forgot Password" functionality with password reset link sent via email.

**FR-1.8**: System shall prevent multiple accounts with the same email address.

**FR-1.9**: System shall hash all passwords using bcrypt before database storage.

**FR-1.10**: System shall enforce role-based permissions restricting classroom creation to Professor role only.

## 4.2 Classroom Management

### 4.2.1 Description and Priority

**Priority: High**

Classroom management enables professors to create private learning spaces and students to join them using unique invite codes. This feature is critical as it establishes the organizational structure for all resource sharing.

### 4.2.2 Stimulus/Response Sequences

**Professor Creates Classroom:**

1. Professor clicks "Create Classroom" on dashboard
2. System displays classroom creation form
3. Professor enters course name, course code, semester, description
4. Professor clicks "Create" button
5. System validates input and generates unique 6-digit invite code
6. System creates classroom in database with professor as owner
7. System displays success message with invite code
8. System redirects to new classroom dashboard

**Student Joins Classroom:**

1. Student clicks "Join Classroom" on dashboard
2. System displays invite code input modal
3. Student enters 6-digit invite code
4. System validates code existence
5. System adds student to classroom member list
6. System displays success message
7. System adds classroom to student's dashboard

### 4.2.3 Functional Requirements

**FR-2.1**: System shall allow only users with Professor role to create classrooms.

**FR-2.2**: System shall generate unique 6-digit numeric invite codes for each classroom.

**FR-2.3**: System shall validate invite codes are unique across entire platform before classroom creation.

**FR-2.4**: System shall require course name and course code as mandatory fields for classroom creation.

**FR-2.5**: System shall set classroom creator as owner with full administrative permissions.

**FR-2.6**: System shall allow students to join classrooms by entering valid invite codes.
**FR-2.7**: System shall display error message if invalid invite code is entered.
**FR-2.8**: System shall prevent duplicate classroom joins (student already member).
**FR-2.9**: System shall allow professors to view classroom member list with roles.
**FR-2.10**: System shall allow professors to remove students from classrooms.
**FR-2.11**: System shall display all enrolled classrooms on user dashboard.
**FR-2.12**: System shall show classroom statistics: total members, total resources, last activity date.

## 4.3 Syllabus Tree Navigation

### 4.3.1 Description and Priority

**Priority: High**

The syllabus tree provides hierarchical organization of course topics, allowing professors to structure curriculum and students to navigate resources efficiently. This feature eliminates folder-based chaos through topic-aligned organization.

### 4.3.2 Stimulus/Response Sequences

1. Professor navigates to classroom
2. Professor clicks "Edit Syllabus" button
3. System displays editable tree structure
4. Professor clicks "Add Unit" button
5. System creates new parent node labeled "Unit 1"
6. Professor enters unit name and description
7. Professor clicks "Add Topic" under unit
8. System creates child node under selected unit
9. Professor drags topics to reorder within tree
10. System saves tree structure changes

### 4.3.3 Functional Requirements

**FR-3.1**: System shall allow professors to create hierarchical syllabus structure with unlimited nesting depth.
**FR-3.2**: System shall support drag-and-drop reordering of syllabus nodes.
**FR-3.3**: System shall display resource count indicators on each syllabus node.
**FR-3.4**: System shall highlight nodes with missing resources in warning color.
**FR-3.5**: System shall allow expand/collapse functionality for syllabus tree navigation.
**FR-3.6**: System shall persist tree expand/collapse state per user.
**FR-3.7**: System shall allow professors to edit node names and descriptions.
**FR-3.8**: System shall allow professors to delete syllabus nodes (with confirmation).

**FR-3.9**: System shall prevent deletion of nodes containing resources (require resource removal first).

**FR-3.10**: System shall display syllabus tree in read-only mode for students.

**FR-3.11**: System shall allow clicking syllabus nodes to filter displayed resources.

## 4.4 File Upload and Management

### 4.4.1 Description and Priority

**Priority: High**

Smart file ingestion enables users to upload educational resources with automatic processing, metadata tagging, and syllabus alignment. This feature is central to resource organization and discovery.

### 4.4.2 Stimulus/Response Sequences

1. User navigates to classroom
2. User clicks "Upload File" button
3. System displays upload modal with drag-and-drop area
4. User drags PDF file into upload area or selects from file picker
5. System validates file type (PDF) and size (under 100MB)
6. System displays metadata form
7. User enters title, description, tags
8. User selects associated syllabus unit and topic from dropdown
9. User clicks "Upload" button
10. System displays progress bar
11. System uploads file to Supabase Storage
12. System performs virus scan on uploaded file
13. System generates thumbnail using first page preview
14. System compresses file if larger than 10MB
15. System saves file metadata to database with pending approval status (if student upload)
16. System sends notification to professor for approval
17. System displays success message
18. System updates file listing and syllabus tree

### 4.4.3 Functional Requirements

**FR-4.1**: System shall support file uploads for PDF, DOCX, PPTX, and TXT formats.
**FR-4.2**: System shall enforce maximum file size of 100MB per upload.
**FR-4.3**: System shall reject file uploads if unsupported format detected.
**FR-4.4**: System shall perform virus scanning on all uploaded files before storage.
**FR-4.5**: System shall delete files immediately if malware detected.
**FR-4.6**: System shall generate thumbnail preview for uploaded files within 10 seconds.
**FR-4.7**: System shall automatically compress files larger than 10MB to optimize storage.
**FR-4.8**: System shall require metadata: title, description, associated syllabus topic.
**FR-4.9**: System shall mark student uploads as "Pending Approval" status.
**FR-4.10**: System shall auto-approve professor uploads without review.
**FR-4.11**: System shall send real-time notification to professor when student uploads file.
**FR-4.12**: System shall allow professors to approve or reject pending uploads.
**FR-4.13**: System shall provide rejection reason input when professor rejects upload.
**FR-4.14**: System shall notify student when upload is approved or rejected.
**FR-4.15**: System shall award 10 karma points to student for approved uploads.
**FR-4.16**: System shall display verification badge on professor-uploaded or professor-approved resources.
**FR-4.17**: System shall support resumable uploads for files larger than 10MB.
**FR-4.18**: System shall allow multiple file selection for batch uploads.
**FR-4.19**: System shall display upload progress percentage in real-time.
**FR-4.20**: System shall allow users to cancel in-progress uploads.

## 4.5 File Version Control

### 4.5.1 Description and Priority

**Priority: Medium**

Version control enables tracking and managing multiple versions of resources, allowing iterative improvements while preserving history. Users can upload new versions and rollback to previous versions.

### 4.5.2 Stimulus/Response Sequences

1. User views existing resource in classroom
2. User clicks "Upload New Version" button on resource card
3. System displays version upload modal pre-filled with existing metadata
4. User selects new file and updates version notes
5. System validates file matches original type
6. User clicks "Upload Version" button
7. System increments version number

8. System archives previous version
9. System sets new file as current version
10. System displays version history with download links for all versions

### 4.5.3 Functional Requirements

**FR-5.1**: System shall maintain version history for all resources.
**FR-5.2**: System shall increment version numbers automatically (v1, v2, v3...).
**FR-5.3**: System shall allow uploading new versions of existing resources.
**FR-5.4**: System shall require version notes explaining changes when uploading new version.
**FR-5.5**: System shall archive previous versions while keeping them accessible.
**FR-5.6**: System shall allow viewing and downloading any historical version.
**FR-5.7**: System shall display version history timeline with upload dates and contributors.
**FR-5.8**: System shall allow professors to rollback to previous versions.
**FR-5.9**: System shall mark current version prominently in file listing.
**FR-5.10**: System shall prevent deletion of resources with multiple versions (must delete all versions).

## 4.6 In-Browser PDF Viewer with Lazy Loading

### 4.6.1 Description and Priority

**Priority: High**
The in-browser PDF viewer eliminates need for downloads, providing seamless document viewing with performance optimization through lazy loading. Users can read, navigate, annotate, and zoom PDFs directly in browser.

### 4.6.2 Stimulus/Response Sequences

1. User clicks on PDF resource thumbnail in classroom
2. System opens full-screen modal PDF viewer
3. System loads first page of PDF using PDF.js
4. System displays page with navigation controls
5. User scrolls down to view next page
6. System lazy-loads page 2 when user approaches bottom of page 1
7. User clicks page thumbnail in sidebar
8. System jumps to selected page and loads it
9. User clicks zoom-in button
10. System increases zoom level and re-renders page

11. User highlights text with mouse drag
12. System captures highlight coordinates and saves to database

### 4.6.3 Functional Requirements

**FR-6.1**: System shall render PDFs using PDF.js library in browser canvas.
**FR-6.2**: System shall load first page of PDF within 1 second of viewer opening.
**FR-6.3**: System shall implement lazy loading, rendering pages only when scrolled into viewport.
**FR-6.4**: System shall pre-load next 2 pages while user views current page.
**FR-6.5**: System shall provide page navigation: next, previous, jump to page number.
**FR-6.6**: System shall display page thumbnail sidebar for quick navigation.
**FR-6.7**: System shall support zoom levels: 50%, 75%, 100%, 125%, 150%, 200%, fit-to-width, fit-to-page.
**FR-6.8**: System shall allow text selection and copying from PDF.
**FR-6.9**: System shall provide download button to save PDF locally.
**FR-6.10**: System shall provide print button triggering browser print dialog.
**FR-6.11**: System shall allow users to highlight text with color selection (yellow, green, blue, pink).
**FR-6.12**: System shall save highlight coordinates to database within 2 seconds of creation.
**FR-6.13**: System shall display user's own highlights when reopening PDF.
**FR-6.14**: System shall allow users to delete their own highlights.
**FR-6.15**: System shall provide fullscreen mode toggle.
**FR-6.16**: System shall allow closing viewer with ESC key or close button.

## 4.7 Collaborative Heatmap Visualization

### 4.7.1 Description and Priority

**Priority: High**

Collaborative heatmaps aggregate student highlights across PDF documents, visualizing popular sections using color gradients. This helps students identify exam-critical content and areas of collective interest.

### 4.7.2 Stimulus/Response Sequences

1. Student opens PDF in viewer
2. Student clicks "Show Heatmap" toggle button
3. System retrieves aggregated highlight data from database
4. System calculates heatmap intensity based on highlight density per page section
5. System renders semi-transparent colored overlay on PDF (light yellow for 1-5 highlights, orange for 6-15, red for 16+)
6. Student hovers mouse over highlighted area
7. System displays tooltip: "23 students highlighted this section"

8. Student navigates to next page
9. System updates heatmap overlay for new page
10. Student adds own highlight in highlighted area
11. System updates heatmap aggregation in real-time (within 5 seconds)

### 4.7.3 Functional Requirements

**FR-7.1**: System shall aggregate highlights from all classroom members for heatmap calculation.

**FR-7.2**: System shall calculate heatmap intensity based on number of overlapping highlights per section.

**FR-7.3**: System shall use color gradient for intensity: light yellow (1-5 highlights), orange (6-15), red (16+).

**FR-7.4**: System shall render heatmap as semi-transparent overlay not obscuring text.

**FR-7.5**: System shall update heatmap within 5 seconds when new highlights are added.

**FR-7.6**: System shall provide toggle button to show/hide heatmap overlay.

**FR-7.7**: System shall display tooltip on hover showing exact highlight count and contributor count.

**FR-7.8**: System shall generate separate heatmap for each page of PDF.

**FR-7.9**: System shall exclude deleted highlights from heatmap calculation.

**FR-7.10**: System shall include only highlights from classroom members (no public highlights).

**FR-7.11**: System shall display "No heatmap data available" message if no highlights exist.

**FR-7.12**: System shall optimize heatmap rendering to not degrade PDF viewer performance.

## 4.8 Discussion Threads

### 4.8.1 Description and Priority

**Priority: Medium**

Discussion threads enable contextual Q&A linked to specific resources and syllabus topics. Students can ask questions, share insights, and professors can provide guidance.

### 4.8.2 Stimulus/Response Sequences

1. Student viewing resource clicks "Start Discussion" button
2. System displays discussion creation form
3. Student enters discussion title and initial question/comment
4. Student optionally tags discussion with syllabus topic
5. Student clicks "Post Discussion" button
6. System creates discussion thread linked to resource

7. System displays thread in classroom discussion panel
8. Other students view discussion and click "Reply" button
9. System displays reply input box
10. User enters reply text and clicks "Post Reply"
11. System adds reply to thread with timestamp
12. System sends notification to thread creator

#### 4.8.3 Functional Requirements

**FR-8.1**: System shall allow users to create discussion threads linked to specific resources.
**FR-8.2**: System shall require discussion title and initial comment as mandatory fields.
**FR-8.3**: System shall allow tagging discussions with syllabus topics.
**FR-8.4**: System shall display discussions in chronological order (newest first).
**FR-8.5**: System shall allow users to reply to discussions with nested comments.
**FR-8.6**: System shall support up to 3 levels of nested replies.
**FR-8.7**: System shall display user name, avatar, and karma level on each comment.
**FR-8.8**: System shall allow users to edit their own comments within 15 minutes of posting.
**FR-8.9**: System shall allow users to delete their own comments (with confirmation).
**FR-8.10**: System shall allow professors to moderate discussions (edit/delete any comment).
**FR-8.11**: System shall send notification to thread creator when new reply is posted.
**FR-8.12**: System shall mark discussions as "Resolved" when professor or creator marks it.
**FR-8.13**: System shall allow upvoting helpful comments (karma +2 for comment author).
**FR-8.14**: System shall display comment count on discussion thread preview.
**FR-8.15**: System shall support basic markdown formatting in discussion text (bold, italic, code).

### 4.9 Exam Resources Section

#### 4.9.1 Description and Priority

**Priority: High**

Dedicated exam sections (Midterm and Endterm) allow students to upload previous year questions (PYQs), important questions, and quick revision notes. Professors supervise and approve uploads to ensure quality.

#### 4.9.2 Stimulus/Response Sequences

1. Student navigates to classroom exam section
2. Student selects "Midterm" or "Endterm" tab
3. Student clicks "Upload Exam Resource" button

4. System displays upload form with resource type dropdown
5. Student selects resource type: PYQ, Important Questions, or Quick Revision
6. Student uploads file and enters metadata (year, topic coverage, description)
7. Student clicks "Submit for Approval" button
8. System validates file and creates database entry with "Pending Approval" status
9. System sends notification to professor
10. Professor reviews upload in pending queue
11. Professor clicks "Approve" button
12. System updates status to "Approved" and awards 15 karma points to student
13. Resource becomes visible to all students in exam section

### 4.9.3 Functional Requirements

**FR-9.1**: System shall provide separate sections for Midterm and Endterm exams.
**FR-9.2**: System shall categorize exam resources into types: PYQ, Important Questions, Quick Revision.
**FR-9.3**: System shall require students to specify resource type during upload.
**FR-9.4**: System shall require year and topic coverage metadata for exam resources.
**FR-9.5**: System shall mark student uploads as "Pending Approval" until professor review.
**FR-9.6**: System shall auto-approve professor uploads in exam section.
**FR-9.7**: System shall send notification to professor when student uploads exam resource.
**FR-9.8**: System shall allow professor to approve or reject pending exam resources.
**FR-9.9**: System shall require rejection reason when professor rejects exam upload.
**FR-9.10**: System shall notify student when exam resource is approved or rejected.
**FR-9.11**: System shall award 15 karma points to student for approved exam uploads.
**FR-9.12**: System shall limit students to maximum 5 uploads per exam section.
**FR-9.13**: System shall allow filtering exam resources by year, type, and topic.
**FR-9.14**: System shall display warning if similar exam resource already exists (duplicate detection).
**FR-9.15**: System shall allow students to edit metadata of pending uploads.

## 4.10 Revision Tests

### 4.10.1 Description and Priority

**Priority: Medium**

Timed revision tests assess student preparation using questions derived from exam materials. System provides immediate feedback, scores, and performance analytics.

#### 4.10.2 Stimulus/Response Sequences

1. Student navigates to exam section and clicks "Revision Tests" tab
2. System displays list of available tests with details (duration, question count, topics)
3. Student selects test and clicks "Start Test" button
4. System displays test instructions and timer
5. Student clicks "Begin" button
6. System starts countdown timer and displays first question
7. Student reads question and selects answer from multiple choices
8. Student clicks "Next" to move to next question
9. System saves answer and displays next question
10. Student completes all questions and clicks "Submit Test"
11. System prompts for confirmation
12. Student confirms submission
13. System stops timer and evaluates all answers against correct answers
14. System calculates score and displays results page with breakdown
15. System shows detailed feedback for each question (correct answer, explanation)
16. System awards karma points based on score percentage
17. System saves test attempt in student's history

#### 4.10.3 Functional Requirements

**FR-10.1**: System shall provide timed revision tests with strict time limits.
**FR-10.2**: System shall display countdown timer during test prominently.
**FR-10.3**: System shall auto-submit test when timer reaches zero.
**FR-10.4**: System shall allow students maximum 3 attempts per test.
**FR-10.5**: System shall randomize question order to prevent cheating.
**FR-10.6**: System shall save answers automatically every 30 seconds.
**FR-10.7**: System shall allow students to navigate between questions during test.
**FR-10.8**: System shall provide "Review Answers" summary before submission.
**FR-10.9**: System shall evaluate answers immediately upon submission.
**FR-10.10**: System shall display results within 2 seconds of submission.
**FR-10.11**: System shall show score as percentage and fraction (e.g., 18/20, 90%).
**FR-10.12**: System shall display correct/incorrect breakdown for each question.
**FR-10.13**: System shall provide explanations for correct answers.
**FR-10.14**: System shall award karma points: 20 (90-100%), 15 (75-89%), 10 (60-74%), 5 (¡60%).
**FR-10.15**: System shall record best score for leaderboard ranking.

**FR-10.16**: System shall maintain test history showing all attempts with scores and dates.
**FR-10.17**: System shall prevent copy-paste during test (disable right-click, select).
**FR-10.18**: System shall warn user before navigating away during active test.
**FR-10.19**: System shall allow resuming incomplete test if browser crashes.
**FR-10.20**: System shall display progress bar showing completion percentage.

## 4.11 Analytics Dashboard (Professor)

### 4.11.1 Description and Priority

**Priority: Medium**

Comprehensive analytics provide professors with data-driven insights into student engagement, resource usage patterns, curriculum gaps, and learning trends.

### 4.11.2 Stimulus/Response Sequences

1. Professor clicks "Analytics" tab in classroom
2. System displays dashboard with overview cards
3. System shows key metrics: total resources, active students, weekly uploads, engagement rate
4. Professor scrolls to view charts:
   - Resource uploads over time (line chart)
   - Most active contributors (bar chart)
   - Topic coverage heatmap (grid showing resource availability)
   - Discussion activity timeline
   - Test performance distribution
5. Professor clicks on topic gap analysis table
6. System highlights syllabus topics with zero or low resources in red
7. Professor clicks "Export Data" button
8. System generates CSV file with detailed analytics
9. System triggers download of analytics report

### 4.11.3 Functional Requirements

**FR-13.1**: System shall display analytics dashboard accessible only to professors.
**FR-13.2**: System shall show overview metrics:

- Total resources count
- Active students (logged in within 7 days)
- Weekly upload count

- Average engagement rate (resource views per student)

**FR-13.3**: System shall display resource upload timeline chart (last 30 days).
**FR-13.4**: System shall show top 10 contributors with upload counts.
**FR-13.5**: System shall generate topic coverage heatmap highlighting gaps.
**FR-13.6**: System shall display discussion activity timeline with comment counts.
**FR-13.7**: System shall show test performance distribution (average scores, pass/fail rates).
**FR-13.8**: System shall highlight syllabus topics with fewer than 3 resources as gaps.
**FR-13.9**: System shall calculate engagement rate as (total resource views / total students).
**FR-13.10**: System shall allow filtering analytics by date range.
**FR-13.11**: System shall provide export functionality for analytics data (CSV format).
**FR-13.12**: System shall refresh analytics data in real-time (update every 60 seconds).
**FR-13.13**: System shall show most viewed resources ranked by view count.
**FR-13.14**: System shall display student activity logs (uploads, views, test attempts).
**FR-13.15**: System shall provide comparison view for different time periods.

# 5 Other Nonfunctional Requirements

## 5.1 Performance Requirements

**NFR-1.1**: Web pages shall load within 2 seconds on standard broadband connection (10 Mbps).

**NFR-1.2**: PDF viewer first page shall render within 1 second of opening.

**NFR-1.3**: System shall support 10+ concurrent users without performance degradation.

**NFR-1.5**: File upload progress shall update in real-time with maximum 1 second delay.

**NFR-1.6**: Database queries shall execute within 100 milliseconds for standard operations.

**NFR-1.7**: Heatmap overlay shall render within 500 milliseconds of toggle activation.

**NFR-1.8**: Search functionality shall return results within 1 second for queries.

**NFR-1.9**: Lazy loading shall load PDF pages within 300 milliseconds of scroll trigger.

**NFR-1.10**: System shall handle file uploads up to 10MB with progress tracking.

**Rationale**: Fast response times are critical for user satisfaction and learning efficiency. Students preparing for exams require quick access to resources.

## 5.2 Safety Requirements

**NFR-2.1**: System shall scan all uploaded files for viruses before storage.

**NFR-2.2**: System shall immediately delete files containing malware and notify user.

**NFR-2.3**: System shall prevent execution of uploaded files on server.

**NFR-2.4**: System shall sanitize all user input to prevent code injection.

**NFR-2.6**: System shall maintain backup retention for 30 days.

**NFR-2.7**: System shall log all critical errors with stack traces for investigation.

**NFR-2.8**: System shall gracefully handle errors without exposing sensitive information.

**NFR-2.9**: System shall validate file integrity using checksums before processing.

**NFR-2.10**: System shall implement rate limiting to prevent denial of service attacks.

**Rationale**: Educational platforms are targets for malware distribution. Safety measures protect students and institutional networks.

## 5.3 Security Requirements

**NFR-3.1**: System shall hash all passwords using bcrypt with salt before database storage.

**NFR-3.2**: System shall enforce HTTPS (TLS 1.2 or higher) for all communications.

**NFR-3.3**: System shall implement JWT-based authentication with 24-hour token expiration.

**NFR-3.4**: System shall validate and sanitize all user inputs to prevent SQL injection.

**NFR-3.5**: System shall implement Cross-Site Request Forgery (CSRF) protection using double-submit cookies.

**NFR-3.6**: System shall enable Cross-Origin Resource Sharing (CORS) only for whitelisted domains.

**NFR-3.7**: System shall implement rate limiting: 10 requests per minute per IP address.

**NFR-3.8**: System shall enforce role-based access control (RBAC) for all operations.
**NFR-3.9**: System shall prevent directory traversal attacks in file access.
**NFR-3.10**: System shall implement Content Security Policy (CSP) headers to prevent XSS attacks.
**NFR-3.11**: System shall encrypt sensitive data at rest in database.
**NFR-3.12**: System shall log all authentication attempts (successful and failed).
**NFR-3.15**: System shall implement secure session management with HTTP-only cookies.
**Rationale**: Educational platforms handle sensitive student data and must comply with privacy regulations.

## 5.4 Software Quality Attributes

### 5.4.1 Usability

**NFR-4.1**: Interface shall be intuitive requiring no prior training for basic operations.
**NFR-4.2**: System shall provide contextual tooltips for all interactive elements.
**NFR-4.3**: Error messages shall be clear, specific, and suggest corrective actions.
**NFR-4.4**: UI shall maintain consistent design patterns across all pages.
**NFR-4.5**: System shall support keyboard navigation for accessibility.
**NFR-4.6**: Loading states shall be indicated with progress indicators or skeleton screens.
**NFR-4.7**: Forms shall validate inputs in real-time with inline error messages.

### 5.4.2 Reliability

**NFR-4.8**: System shall maintain 99.5% uptime during semester periods.
**NFR-4.9**: System shall recover from failures within 5 minutes.
**NFR-4.10**: System shall handle concurrent edits with conflict resolution (last-write-wins).
**NFR-4.11**: System shall maintain data consistency across all transactions.

### 5.4.3 Scalability

**NFR-4.12**: System architecture shall support horizontal scaling for increased load.
**NFR-4.13**: Database shall handle 1000+ resources per classroom efficiently.
**NFR-4.14**: System shall support 100+ concurrent PDF viewers without degradation.
**NFR-4.15**: Cloud storage shall scale automatically to accommodate growing file uploads.

### 5.4.4 Maintainability

**NFR-4.16**: Code shall follow ESLint standards for consistency.
**NFR-4.17**: System shall use modular architecture for easy component updates.
**NFR-4.18**: API shall be versioned for backward compatibility.
**NFR-4.19**: System shall include comprehensive logging for debugging.

### 5.4.5 Portability

**NFR-4.20**: Application shall run on all major browsers without modification.
**NFR-4.21**: Responsive design shall adapt to screen sizes from 320px to 4K displays.
**NFR-4.22**: System shall function on desktop and mobile platforms.

### 5.4.6 Interoperability

**NFR-4.23**: System shall provide RESTful API for potential future integrations.
**NFR-4.24**: Data exports shall use standard formats (CSV, JSON).
**NFR-4.25**: System shall support standard file formats (PDF, DOCX, PPTX, TXT).

## 5.5 Business Rules

**BR-1**: Only users with verified university email addresses may register.
**BR-2**: Only professors may create classrooms and manage syllabus structure.
**BR-3**: Student uploads require professor approval before becoming visible to classroom.
**BR-4**: Professor uploads are auto-approved and receive verification badge.
**BR-5**: Invite codes must be exactly 6 digits and unique across platform.
**BR-6**: Maximum file size for uploads is 100MB.
**BR-7**: Supported file types are PDF, DOCX, PPTX, TXT only.
**BR-11**: Only classroom members can access classroom resources.
**BR-12**: Heatmap aggregates highlights only from classroom members.
**BR-13**: Users can only edit or delete their own content (except professors).
**BR-14**: Professors have moderation rights over all classroom content.
**BR-15**: Version history must be preserved; previous versions cannot be deleted individually.

# 6 Other Requirements

## 6.1 Database Requirements

**DB-1**: System shall use PostgreSQL 14.x or higher for relational data storage.

**DB-2**: Database shall implement referential integrity with foreign key constraints.

**DB-3**: Database shall enforce unique constraints on: user emails, classroom invite codes, resource IDs.

**DB-4**: Database shall use indexes on frequently queried columns (user_id, classroom_id, resource_id).

**DB-5**: Database shall implement ACID compliance for all transactions.

**DB-6**: Database connection pool shall maintain minimum 10 connections for performance.

## 6.2 Internationalization Requirements

**I18N-1**: System shall display timestamps in user's local timezone.

**I18N-2**: System shall use UTF-8 encoding for all text storage and display.

**I18N-3**: Future versions may support multiple languages (English as base language).

## 6.3 Legal Requirements

**LEGAL-1**: System shall include Terms of Service agreement during registration.

**LEGAL-2**: System shall include Privacy Policy accessible from footer.

**LEGAL-3**: Users shall agree to copyright policy before uploading content.

**LEGAL-4**: System shall provide mechanism for DMCA takedown requests.

**LEGAL-5**: System shall retain user data only as long as account is active plus 90 days.

## 6.4 Installation and Deployment Requirements

**DEPLOY-1**: System shall support containerized deployment using Docker.

**DEPLOY-2**: System shall include environment configuration for development, staging, production.

**DEPLOY-3**: System shall use CI/CD pipeline for automated testing and deployment.

**DEPLOY-4**: System shall support zero-downtime deployments.

**DEPLOY-5**: System shall include health check endpoints for monitoring.

# Appendix A: Glossary

**Classroom** Private learning space created by professor where resources are shared among enrolled students.

**Heatmap** Visual overlay on PDF showing areas highlighted by multiple students using color gradients.

**Invite Code** Unique 6-digit numeric code used by students to join classrooms.

**Lazy Loading** Performance optimization loading PDF pages only when scrolled into view.

**Neon DB** Managed PostgreSQL database service used for data persistence.

**PDF.js** JavaScript library for rendering PDF documents in browser without plugins.

**PYQ** Previous Year Question paper uploaded to exam section.

**RBAC** Role-Based Access Control distinguishing Professor and Student permissions.

**Supabase** Backend-as-a-Service platform providing authentication and file storage.

**Syllabus Tree** Hierarchical curriculum structure organizing resources by units and topics.

**Version Control** System tracking multiple versions of same resource with history.

# Appendix B: Analysis Models

## Data Flow Diagram - Level 1

The NoteFlow system contains the following main subsystems:

**User Authentication**: Handles registration, login, email verification, and session management using JWT tokens.

**Classroom Management**: Manages classroom creation, invite code generation, and membership management.

**File Management**: Processes file uploads with virus scanning, thumbnail generation, compression, and metadata tagging.

**Syllabus Tree**: Maintains hierarchical course structure with drag-and-drop editing capabilities.

**Heatmap Engine**: Aggregates student highlights and generates visual intensity overlays.

**Discussion Forum**: Enables threaded discussions linked to resources and topics.

**Analytics Dashboard**: Provides professors with insights into engagement patterns and resource usage.

All subsystems interact with the central PostgreSQL database and Supabase storage services. Data flows between components via RESTful API calls secured with JWT authentication.

## Entity-Relationship Model

**Key Entities and Relationships:**

- **User** (1) creates (M) **Classroom**
- **User** (M) joins (M) **Classroom**
- **Classroom** (1) has (M) **Syllabus Node**
- **Syllabus Node** (1) contains (M) **Resource**
- **User** (1) uploads (M) **Resource**
- **Resource** (1) has (M) **Version**
- **User** (1) creates (M) **Highlight**
- **Resource** (1) has (M) **Highlight**
- **User** (1) posts (M) **Discussion**
- **Resource** (1) has (M) **Discussion**

# Appendix C: To Be Determined List

1. **TBD-1**: Final selection of virus scanning API service (ClamAV vs VirusTotal)
2. **TBD-2**: Specific cloud hosting platform (Vercel vs Netlify vs AWS)
3. **TBD-3**: Email service provider for notifications (SendGrid vs AWS SES)
4. **TBD-4**: Mobile application development timeline and framework (React Native vs Flutter)
5. **TBD-5**: AI summarization feature implementation approach and API
6. **TBD-6**:Heatmap support feasibility
7. **TBD-7**: LMS integration protocols and university system APIs
8. **TBD-8**: Maximum number of classrooms per professor account
9. **TBD-9**: Data retention policy after course semester completion
10. **TBD-10**: Proctoring features for revision tests (webcam monitoring)
11. **TBD-11**: Advanced analytics metrics and machine learning recommendations
12. **TBD-12**: Multi-language support priority languages and timeline